Disjoint-set data structure

Union-find data structure

Adapted for CompO by **Kamonphop Srisopha**

Disjoint Set In Math

- ในทางคณิตศาสตร์ เซตสองเซตจะเรียกว่า **เซตไม่ทับซ้อนกัน (disjoint sets)** หากทั้งสองเซต **ไม่มีสมาชิกใดร่วมกันเลย**

- กล่าวอีกอย่างหนึ่งคือ เซตไม่ทับซ้อนกันคือเซตที่มี **ส่วนร่วม (intersection) เป็นเซตว่าง**

- ตัวอย่างเช่น {1, 2, 3} และ {4, 5, 6} เป็นเซตไม่ทับซ้อนกัน แต่ {1, 2, 3} และ {3, 4, 5} **ไม่เป็น**เซตไม่ทับซ้อนกัน เพราะมีสมาชิกร่วมกันคือ 3

Quick Example

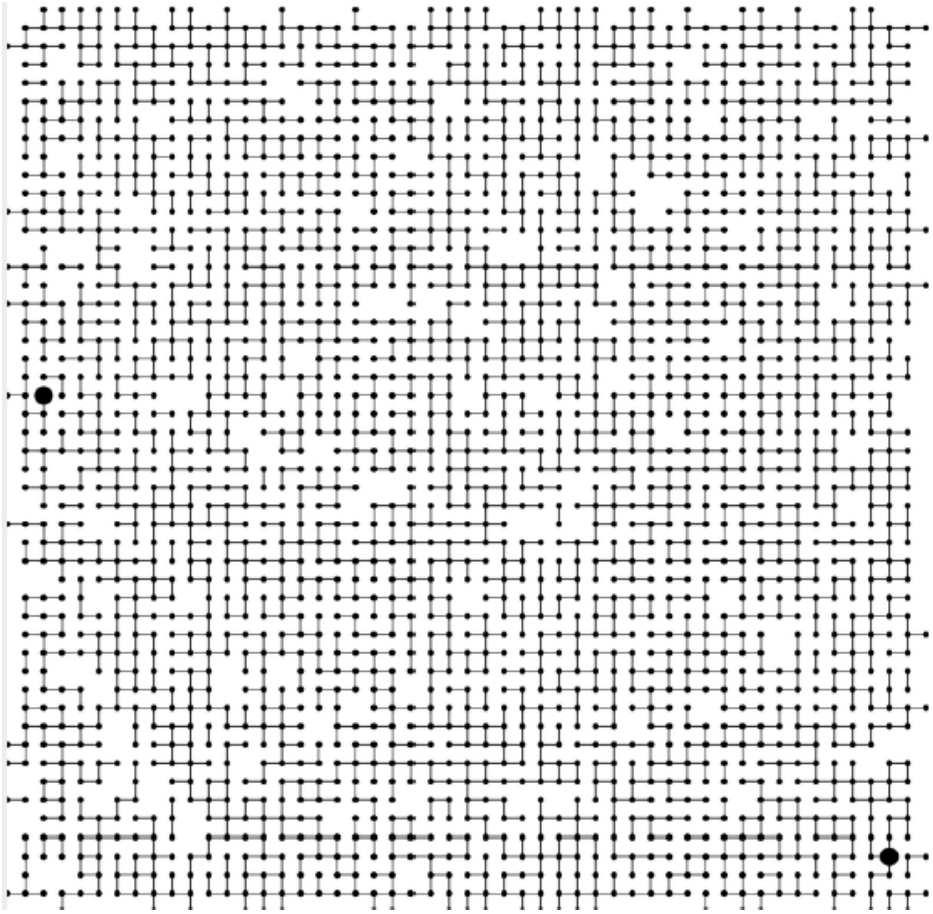
- พิจารณาสถานการณ์ที่มีบุคคลจำนวนหนึ่ง และมี task ต่อไปนี้ที่ต้องทำกับบุคคลเหล่านั้น

- เพิ่มความสัมพันธ์แบบเพื่อนใหม่ กล่าวคือ บุคคล i กลายเป็นเพื่อนกับบุคคล j
- ตรวจสอบว่าบุคคล k เป็นเพื่อนกับบุคคล j หรือไม่ (เป็นได้ทั้งเพื่อนโดยตรง หรือเพื่อนทางอ้อม)

Example: Network Connectivity

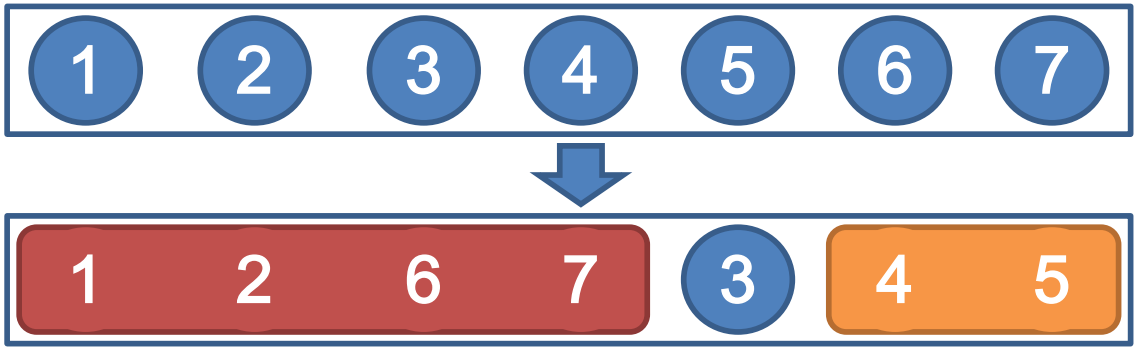
ตัวอย่างเมื่อพิจารณาปัญหาการเชื่อมต่อกัน

- พิจารณา set ของวัตถุ
- **union** operation:
  - เชื่อมสอง Disjoint Set เข้าด้วยกัน
- **find** operation:
  - วัตถุที่สอบถามอยู่ใน Set ไหน
  - วัตถุที่สอบถามเชื่อมต่อกันไหม
  - มี path ที่เชื่อมระหว่างวัตถุที่สนใจหรือไม่ (อยู่กลุ่มเดียวกันไหม)



Union-Find Disjoint Sets

- โครงสร้างข้อมูลนี้สามารถถูกนำไปใช้ในปัญหาเกี่ยวกับการหา **connected component** ใน undirected graph
- เริ่มต้นที่ตั้งให้ว่าแต่ละ node เป็น disjoint set
- หลังจากนั้นเราจะทำการรวมกลุ่มของ node **หาวิธีเชื่อมกัน**
- ดังนั้นเราสามารถทดสอบว่าสอง node อยู่ใน set เดียวกันโดยการถามหา**ตัวแทน**ได้ก็จะแก้ปัญหาได้ไม่ยาก



ข้อจำกัดต่าง ๆ

- C++ STL set ถูกออกแบบมาสำหรับการเก็บข้อมูลแบบเรียงลำดับ ไม่ได้ออกแบบมาสำหรับปัญหา **disjoint set** โดยตรง

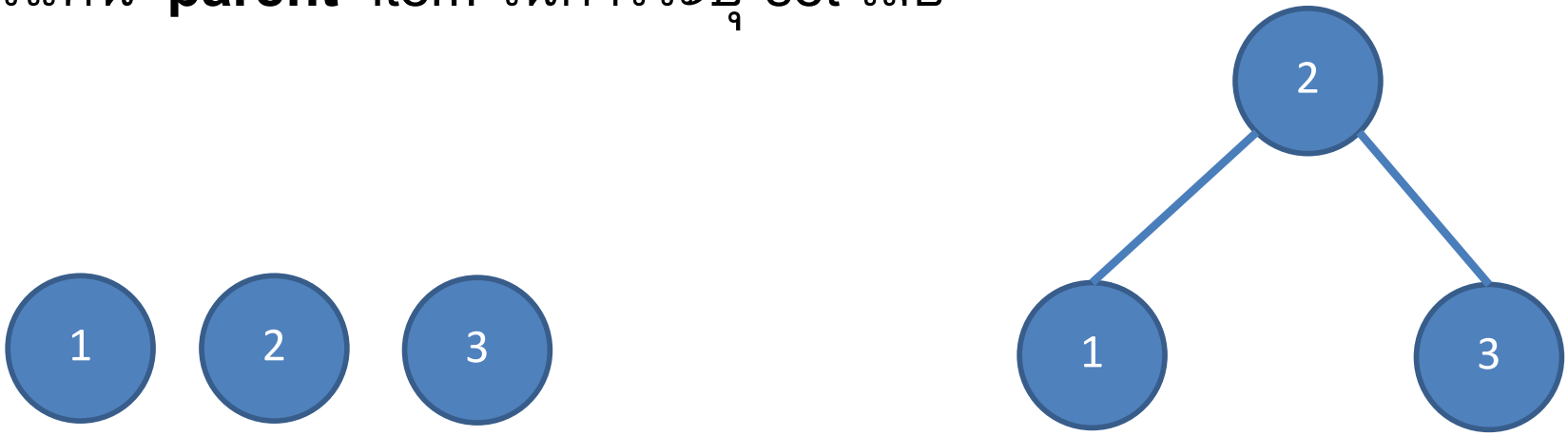
- ถ้าใช้ vector<set> จะต้องวนหาว่า item อยู่ใน set ใด ⟶ การทำ find **ไม่มีประสิทธิภาพ**

- การรวมสอง set ด้วย C++STL set_union ต้องไล่สมาชิกของทั้งสอง set ⟶ **O(n)** เมื่อมีการ find และ union บ่อย ๆ วิธีนี้จึงไม่มีประสิทธิภาพ

- ดังนั้นจึงใช้ **Union-Find (Disjoint Set Union)** ซึ่งออกแบบมาเพื่อรองรับงานนี้โดยเฉพาะนั้นคือ

- แนวคิดของโครงสร้างข้อมูลนี้คือการ**เลือกตัวแทน** item (parent) เป็นตัวแทนของ set

- ถ้าเราแน่ใจว่าแต่ละ set ถูกแทนด้วย item เพียงตัวเดียว แล้วการตัดสินใจว่า item นั้นอยู่ใน set หรือไม่ จะทำได้ง่ายเลย นั่นคือ เราใช้ตัวแทน **'parent'** item ในการระบุ set เลย



Implementation?

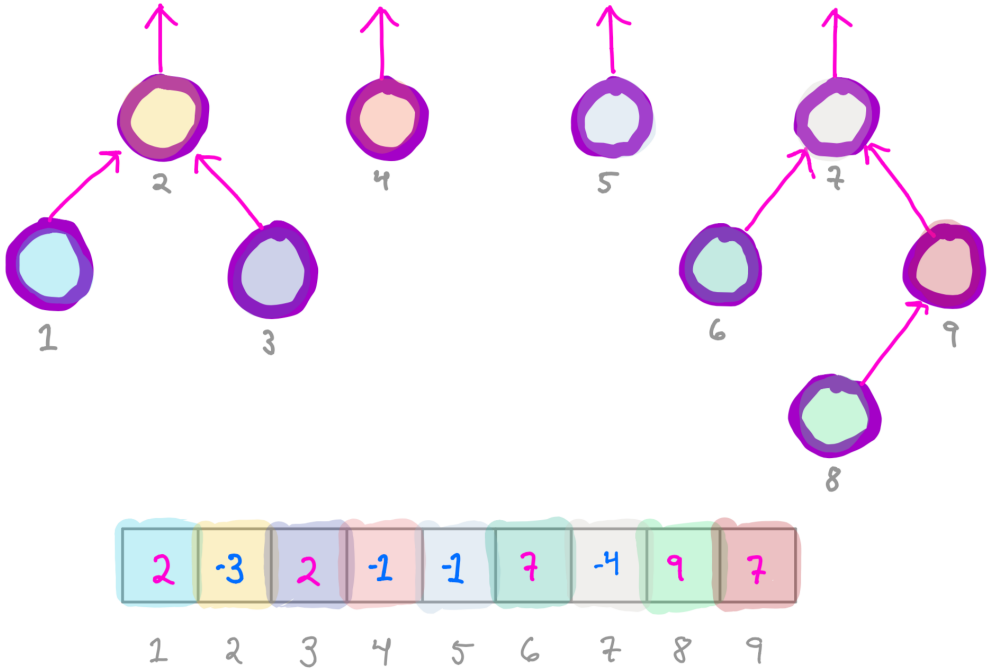
- ในการสร้าง Union-Find Disjoint Set ใช้โครงสร้างอะไรที่หาตัวแทนง่าย?

- จะสร้างโครงสร้าง **tree** ที่ disjoint sets นั้นอยู่ในรูปของต้นไม้หลายๆ ต้น (Forest of trees) โดยต้นไม้แต่ละต้นนั้นจะสอดคล้องกับ 1 disjoint set ทั้งนี้ **root** ของ tree จะถูกนำมาใช้เป็นตัวแทนของ set

- ตัวแทนที่เป็นตัวบ่งบอก set นั้นสามารถทำได้โดยการ traverse tree จาก node หนึ่ง ๆ **ไปยัง root** ของ tree เนื่องจาก tree มีได้เพียง 1 root

Array Implementation

- เราสามารถใช้ Array ในการเขียน Disjoint Set ได้
  - ใช้ Index ของ Array แทนข้อมูล
  - ค่าของ Array ที่ Index (arr[i]) คือค่าที่ Pointer ชี้ไป ถ้า Index เป็นตัวแทนของกลุ่ม ค่าที่ตำแหน่งนั้นจะเป็นเลขลบ (หรือใช้เป็นค่า index ของตัวเอง)
  - ตัวอย่าง



Union Find

- อัลกอริทึม **union-find** คืออัลกอริทึมที่ใช้ดำเนินการหลัก 2 อย่างบนโครงสร้างข้อมูล **disjoint-set** ดังนี้

- **Find:** ใช้ตรวจสอบว่าสมาชิกตัวหนึ่งอยู่ในเซตย่อยใด ซึ่งสามารถนำไปใช้ตรวจสอบได้ว่าสมาชิก 2 ตัวอยู่ในเซตย่อยเดียวกันหรือไม่

- **Union:** ใช้รวมเซตย่อย 2 เซตให้กลายเป็นเซตย่อยเดียวกัน

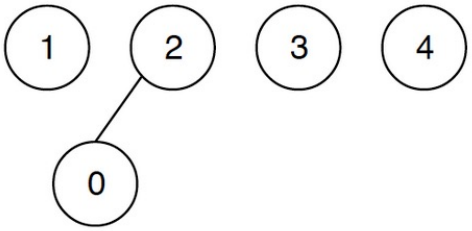
Union Find

Union Find

a) Initial

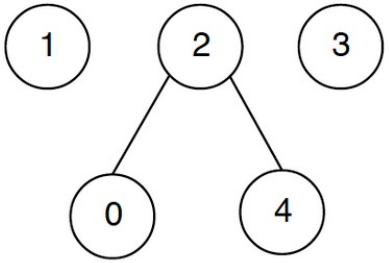
parent

0	1	2	3	4



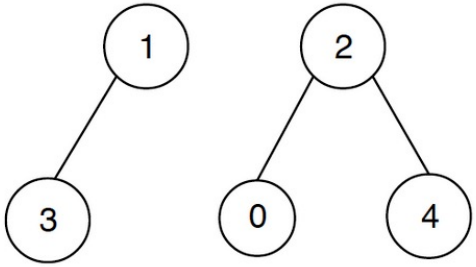
b) Union(0, 2)

2	1	2	3	4



c) Union(4, 2)

2	1	2	3	2



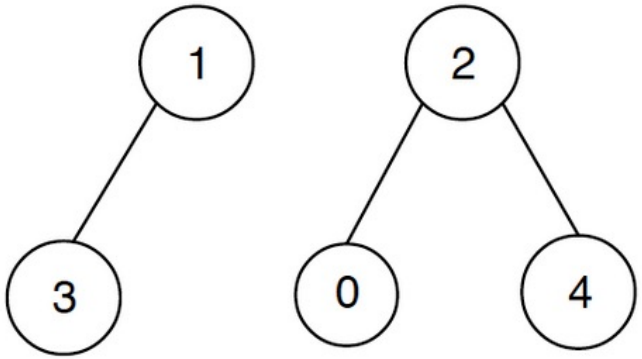
d) Union(3, 1)

2	1	2	1	2

https://jojozhuang.github.io/algorithm/algorithm-union-find/

Find

- ถ้าเราอยากจะรู้ว่า 0 และ 4 อยู่ใน set เดียวกันหรือไม่ เราก็แค่ใช้ find(0) และ find(4) แล้วดูว่า traverse ไปเจอ parent เดียวกันมั้ย
- แล้วถ้า
  - find(3)
  - find(4)



2	1	2	1	2

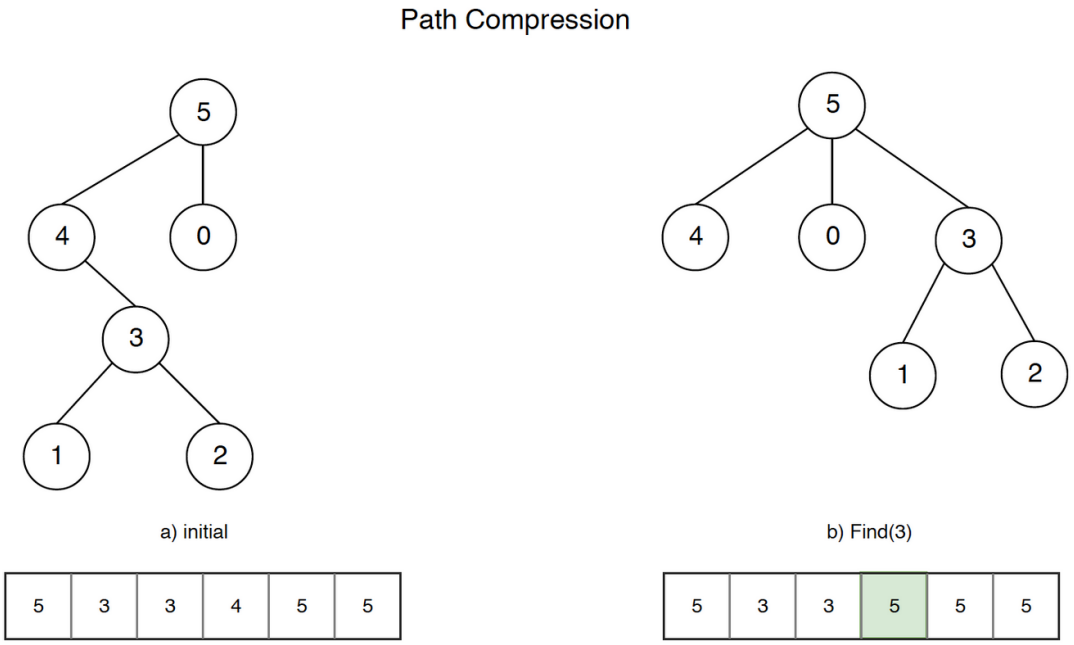
Find

```
int find(int x) {
    if (parent[x] == x) {
        return x;   // root
    }
    return find(parent[x]);
}
```

คิดว่า implementation ของ find นี้โอเคหรือยัง? ถ้ายังมันมีปัญหาตรงไหน?

Path Compression

- ตอนเรียกฟังก์ชัน Find เราสามารถย้าย node ที่อยู่ในเส้นทางค้นหาให้ไปชี้ที่ Root หมดได้ การย้าย node นี้ไม่ได้เพิ่ม Time Complexity แต่อย่างใด ในทางกลับกันจะทำให้การเรียก Find บน node เหล่านั้นเร็วขึ้นอีกในอนาคต



ถ้าเราเรียก find(3) ซ้ำๆ แบบไม่มี Path Compression เราต้อง traverse tree ขึ้นไปหา root ทุกครั้ง

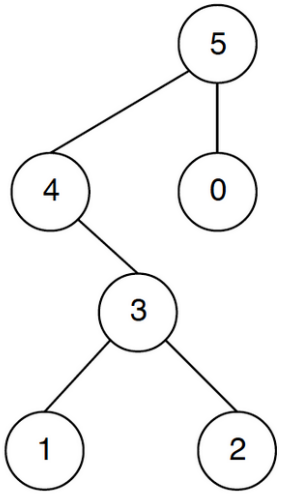
แต่ถ้า find(3) เรามี path compression เมื่อเรียกเสร็จครั้งแรก parent ของ 3 ก็จะถูกอัปเดตเป็น root โดยอัตโนมัติ การเรียก find(3) ครั้งต่อไปก็จะเร็วมาก (ไม่ต้อง traverse tree ใหม่)

Find with Path Compression

```
int find(int x) {
    if (parent[x] == x) {
        return x;   // root
    }
    parent[x] = find(parent[x]);   // path compression
    return parent[x];
}
```

Path Compression

- ตอนเรียกฟังก์ชัน Find เราสามารถย้าย node ที่อยู่ในเส้นทางค้นหาให้ไปชี้ที่ Root หมดได้ การย้าย node นี้ไม่ได้เพิ่ม Time Complexity แต่อย่างใด ในทางกลับกันจะทำให้การเรียก Find บน node เหล่านั้นเร็วขึ้นอีกในอนาคต



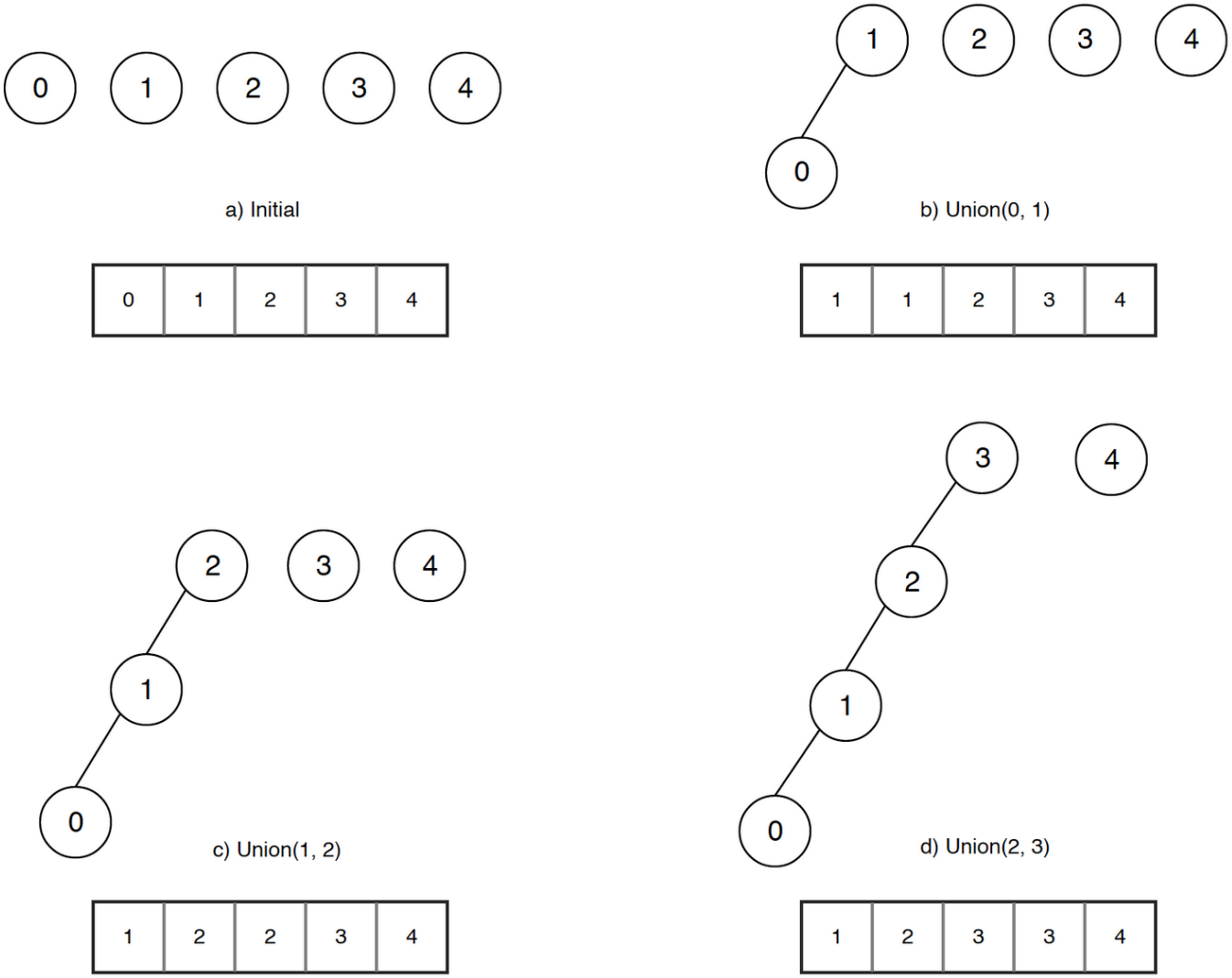
a) initial

5	3	3	4	5	5

ถ้าเราเรียก find(2), ผลลัพธ์ของ Array จะเป็นอย่างไร?

ปัญหาต่อมา

Naive Union



เวลาทำ Union เราจะเลือก node ไหนเป็น parent?

ถ้า Union แบบไม่มี Strategy ที่ดีก็อาจจะเกิดปัญหาแบบนี้ (เช่น Union โดยการดูที่ index เอาน้อยไปต่อมาก)

ถ้าเราเรียก find(0) เราก็ต้อง traverse จาก node 0 ไป node 4 ซึ่งไม่มีประสิทธิภาพ

https://jojozhuang.github.io/algorithm/algorithm-union-find/

การจะทำให้มีประสิทธิภาพ

สอง Options ที่จะทำให้การ Union แล้วผลลัพธ์มีประสิทธิภาพยิ่งขึ้น

Union by Rank (ความสูงของ tree)

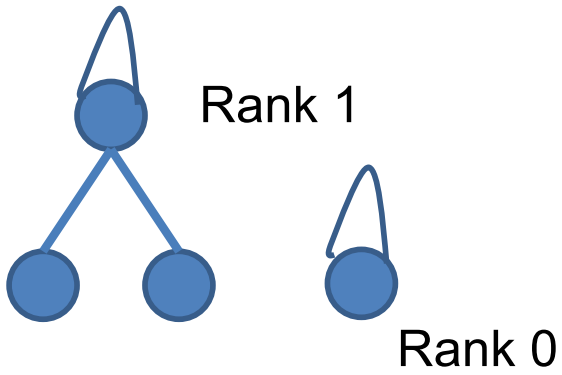
Union by Size (จำนวนของ node ใน set)

การจะทำให้มีประสิทธิภาพ

- **เราจะเก็บ index ของ parent** และ**ความสูงของ tree ของแต่ละ set** ไว้ (vi p และ vi rank)
  ทั้งนี้ vi คือ vector of integer p[i] เก็บ parent ของ item i

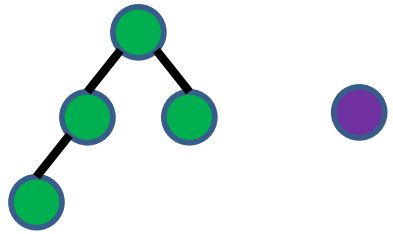
- ถ้า item i เป็นตัวแทนของ disjoint set แล้ว **p[i] = i** นั้นคือ **self-loop**

- **rank[i]** เก็บ**ความสูง**ของ tree ที่มี root ที่ item i



กำหนดตัวแทนตัวไหนดี

- ในการทำให้มีประสิทธิภาพ เราจะใช้ข้อมูลใน **vi rank** เพื่อกำหนดตัวแทน item ของตัวที่มี **rank สูงกว่าให้เป็น parent** ใหม่ของอีกตัวที่มี rank ต่ำกว่า เพื่อทำให้ rank ของ tree ที่ได้ต่ำที่สุด



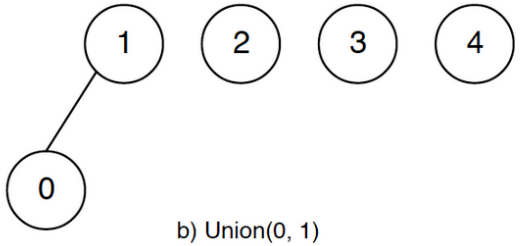
- นั่นคือต้นที่เตี้ยกว่าจะมาต่อต้นที่สูงกว่า ทำไมถึงดี?
- เริ่มต้นเราจะให้แต่ละ item เป็น disjoint set ด้วยตัวมันเองและให้ rank = 0 และ parent ของแต่ละตัวเป็นตัวมันเอง
- ถ้าทั้งคู่มี rank เท่ากัน เราจะเลือกมาสักตัวหนึ่งให้เป็น parent ใหม่และเพิ่มค่าของ root's rank ทำแบบนี้เรียกว่า **union by rank**

Union By Rank

Union by Rank

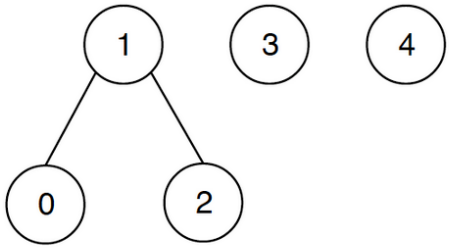
a) Initial

parents	0	1	2	3	4
rank	0	0	0	0	0



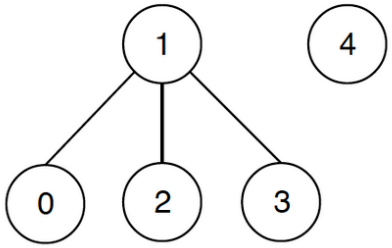
b) Union(0, 1)

parents	1	1	2	3	4
rank	0	1	0	0	0



c) Union(1, 2)

parents	1	1	1	3	4
rank	0	1	0	0	0

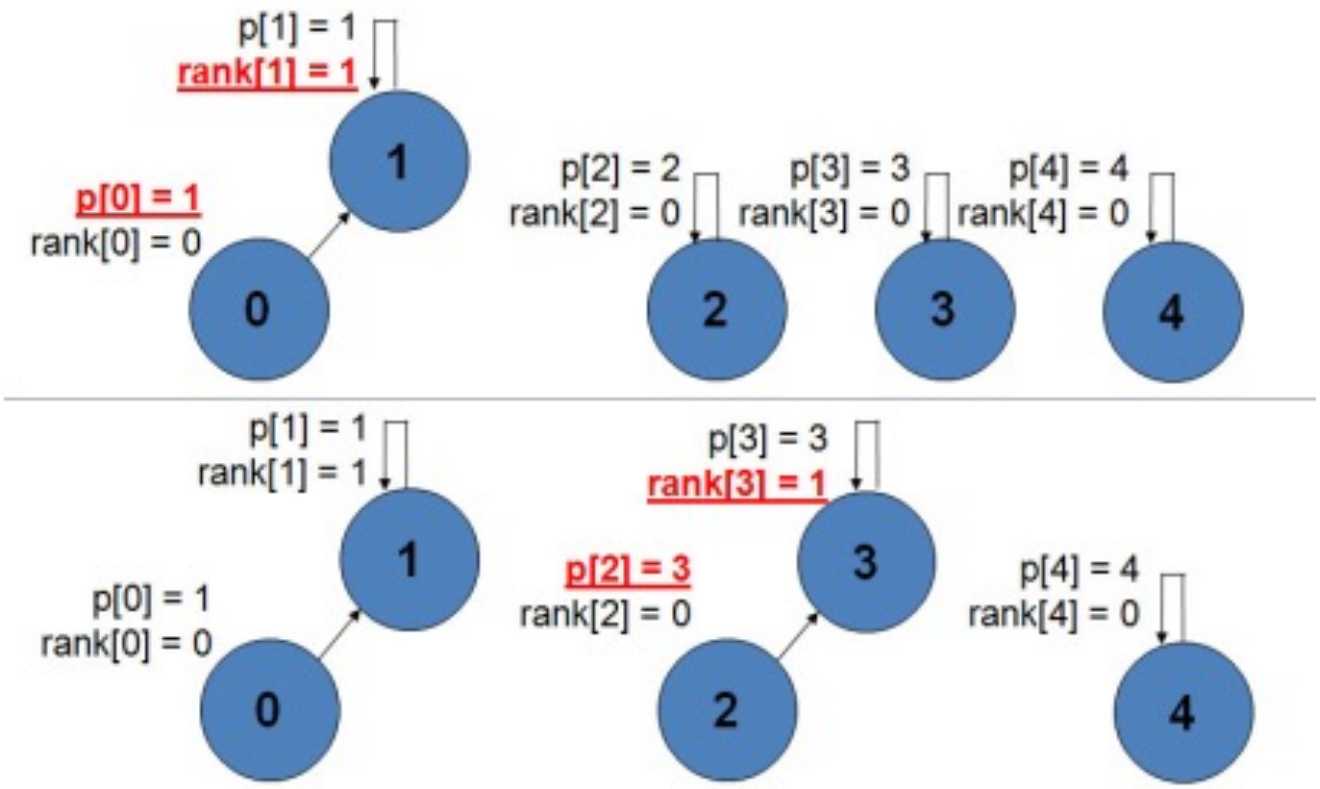


d) Union(2, 3)

parents	1	1	1	1	4
rank	0	1	0	0	0

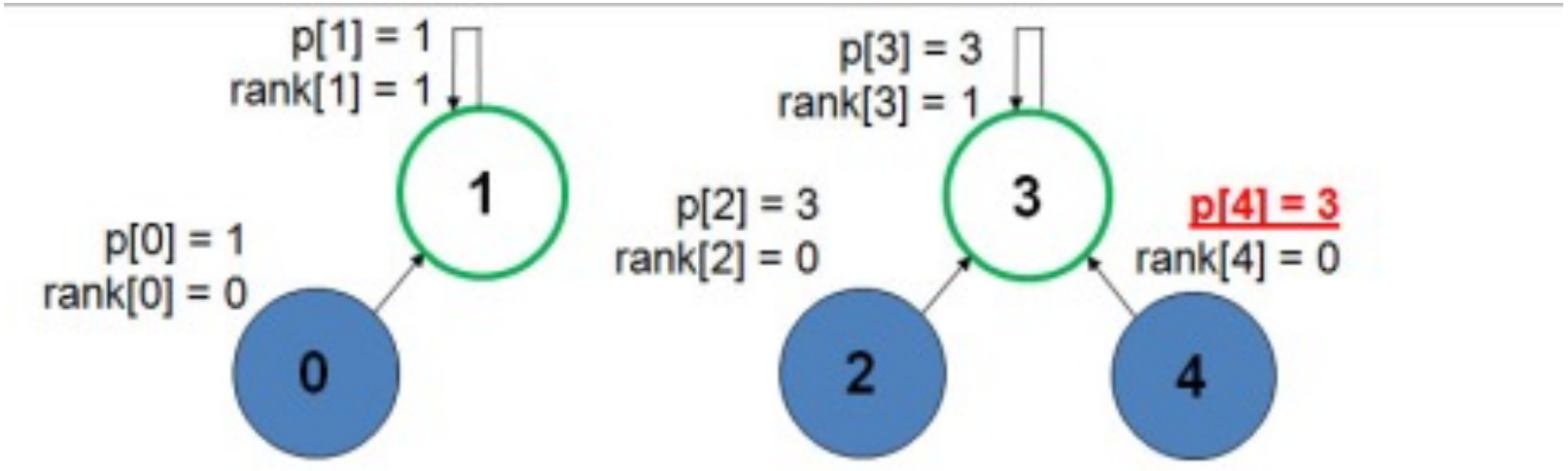
https://jojozhuang.github.io/algorithm/algorithm-union-find/

Another Example

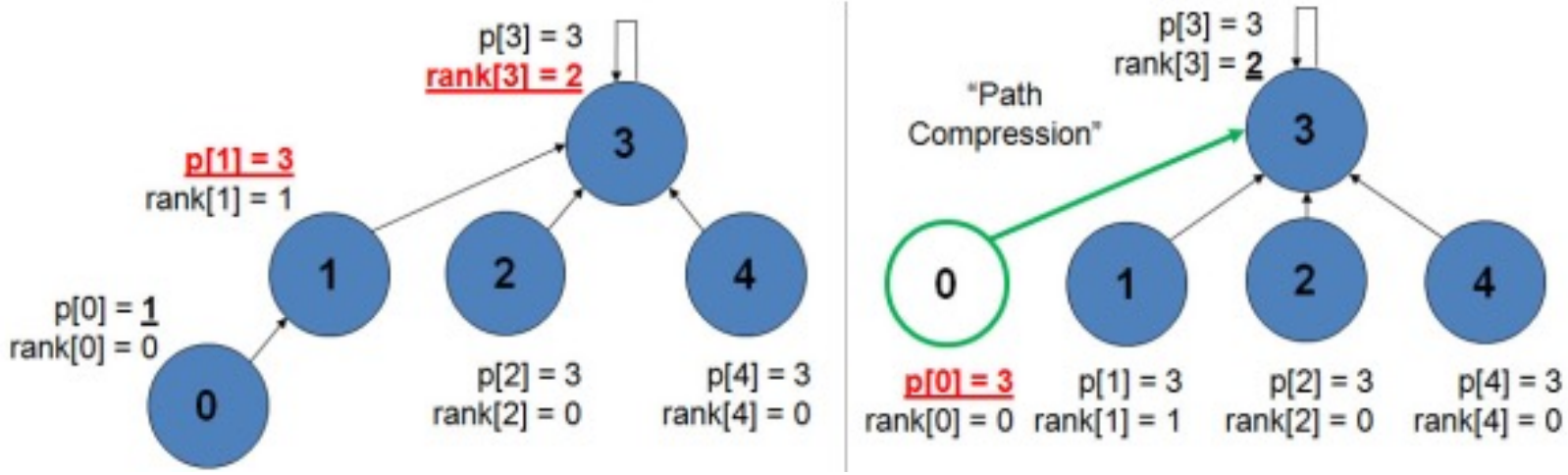


- รูปต่อไปเป็นตัวอย่าง unionSet(0,1) โดยกำหนดให้ p[0] เป็น 1 และ rank[1] เป็น 1 และต่อไปเป็น unionSet(2,3) โดยกำหนด p[2] = 3 และ rank[3]=1

- ฟังก์ชัน find(i) จะเรียก find(p[i])แบบ **recursive** เพื่อหาตัวแทนของ set
- ในรูปต่อไป มีการเรียก unionSet(4,3) เรามี rank[find(4)] = rank[4]=0 ซึ่งน้อยกว่า rank[find(3)] = rank[3]=1
- ดังนั้นเมื่อเรากำหนด p[4]=3 โดย**ไม่**เปลี่ยนแปลงความสูงของ tree
- นี่คือการทำงานของ union by rank



- ทีนี้หากต้องการตรวจสอบว่าอยู่ set เดียวกันหรือไม่ isSameSet(0,4) ทำงานได้โดยไปเรียกหาว่า find(0) และ find(4) เป็นตัวเดียวกันหรือไม่
- ตัวอย่างเช่น find(4) = find(p[4])= find(3)=3

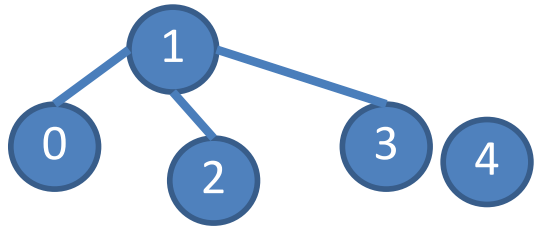


- ถ้าเรา find หา root แล้วเราก็ set parent เป็น root เลยนั่นคือ **path compression**

```
#include <bits/stdc++.h>
using namespace std;
typedef vector<int> vi;
// Union-Find Disjoint Sets Library
// 1000 ปรับได้
vi p(1000), ranks(1000);
void initSet(int N) {
        ranks.assign(N, 0);
        p.assign(N, 0);
        for (int i = 0; i < N; i++)
                p [i] = i;
}
```

```
int findSet(int i) {
  return (p[i] == i) ? i : (p[i] = findSet(p [i]));}

bool isSameSet(int i, int j) {
  return findSet(i) == findSet(j);}

void unionSet(int i, int j) {
  //implement unionSet by Rank





}
```

```c
int main() {
  printf("Assume that there are 5 disjoint sets initially\n");
  initSet(5); // create 5 disjoint sets
  unionSet('A' - 'A', 'B' - 'A'); // unionSet(A, B)
  unionSet('A' - 'A', 'C' - 'A'); // unionSet(A, C)
  unionSet('D' - 'A', 'B' - 'A'); // unionSet(D, B)
  printf("find(A) = %d\n", find('A' - 'A'));
  // will return 1
  printf("find(B) = %d\n", find('B' - 'A'));
  // will return 1
  printf("find(C) = %d\n", find('C' - 'A'));
  // will return 1
  printf("find(D) = %d\n", find('D' - 'A'));
  // will return 1
  printf("find(E) = %d\n", find('E' - 'A'));
  // will return 4 (the parent ID of 'E')
  printf("isSameSet(A, E) = %d\n", isSameSet('A' - 'A', 'E' - 'A'));
// will return 0 (false)
  printf("isSameSet(A, B) = %d\n", isSameSet('A' - 'A', 'B' - 'A'));
// will return 1 (true)
  return 0;
}
```



Problem: General Online

- ผู้เล่นมีสองระดับคือ นายพล และ เชลย (ของนายพลคนอื่น) โดยเริ่มต้นที่ระดับ นายพล และมีทหารอยู่จำนวนนึง
- เมื่อผู้เล่นสองคนสู้กัน
  - ถ้าผู้เล่นเป็น เชลย นายพลของเชลยนั้นจะมาสู้แทน
  - ผู้เล่นที่มีกำลังพลมากกว่าชนะ ผู้แพ้จะกลายเป็น เชลย ของผู้ชนะ และผู้ชนะได้ทหารครึ่งนึงของผู้แพ้ (ปัดเศษทิ้ง)
  - ถ้ามีกำลังพลเท่ากัน ผู้เล่นที่มีหมายเลขน้อยกว่าเป็นผู้ชนะ (เล่นก่อนได้เปรียบ)
  - ถ้าเชลยของนายพลสู้กันเอง จะไม่มีผลแพ้ชนะ
- มี Log File ของการสู้กันมาให้ จงแสดงว่าฝ่ายไหนชนะ

Example

ID: 1
นายพล
ทหาร: 3

ID: 2
นายพล
ทหาร: 4

ID: 3
นายพล
ทหาร: 5

ID: 1
เชลยของ 2
ทหาร: -

ID: 2
นายพล
ทหาร: 5

รอบแรก [1 vs 2]
2 ชนะเพราะทหารเยอะกว่า
1 กลายเป็นเชลยของ 2
ทหารของ 2 เพิ่มเป็น 4 + 3/2 = 5

ID: 1
เชลยของ 2
ทหาร: -

ID: 2
นายพล
ทหาร: 7

ID: 3
เชลยของ 2
ทหาร: -

รอบสอง [1 vs 3]
1 เป็นเชลยของ 2 ดังนั้น 2 มาสู้แทน
2 ชนะเพราะ ID น้อยกว่า แม้ทหารเท่ากัน
3 กลายเป็นเชลยของ 2
ทหารของ 2 เพิ่มเป็น 5 + 5/2 = 7